

ways. This is why consistent monitoring plays such a crucial role in machine learning. When performance dips, the cycle returns to the beginning: collect updated data, retrain, and release an improved version.

This continuous loop of data, development, deployment, and monitoring is what defines the ML project lifecycle. Each step affects the next, and even a small error can ripple through the system if not caught early. When this lifecycle is supported only by manual work, it becomes slow and prone to mistakes. Automating parts of it with CI/CD allows teams to catch issues sooner and ship better models faster. Instead of worrying whether changes might break the system, developers build confidently, knowing that every update is validated and traceable. In a world where models must grow and adapt, CI/CD enables the ML lifecycle to keep up with the pace of change.

Setting up the ML project repository

Before building automation or deploying a model, the foundation of a machine learning project must be well-organised. A properly structured repository not only makes development easier but also ensures that CI/CD pipelines can run smoothly without confusion about where data, models, or scripts are located. Many beginners start with everything inside a single notebook, but as a project progresses, that approach becomes difficult to maintain or share within a team.

A clean project structure usually separates code, data, and trained models into different folders. Scripts for preprocessing, feature engineering, and model training live inside a dedicated source directory, while configuration files and dependencies are defined so they can be reproduced on any machine. This is especially important for CI workflows, which install everything fresh each time they



Figure 2: Understanding the ML project lifecycle

run. When the structure is simple and predictable, automation becomes far easier to build, test, and trust.

Machine learning projects often work with large datasets and binary model files that cannot be pushed directly to GitHub because of size limitations. Tools such as Git LFS, which stands for Large File Storage, help manage these heavy assets without slowing down the repository or cluttering version history. Storing models in a structured and traceable way ensures that every version used during training or deployment is clearly identified. This becomes especially important when issues arise in production, because teams must be able to quickly trace back to the exact model version, review how it was produced, and roll back if necessary.

Proper versioning and artifact handling maintain control as models evolve, allowing development to remain smooth, transparent, and reliable over time. Even the way work is pushed matters. Following a branch strategy, such as using separate branches for experimentation and final production changes, keeps the project clean and prevents half-complete features from accidentally going live. Review processes, pull requests, and version tags all contribute to a more controlled development environment.

With a well-designed repository in place, the next steps in the CI/CD pipeline become more reliable. The

project is prepared for automation, collaboration, and long-term maintenance. Good organisation may feel like a simple step, but it sets the stage for everything else that follows.

Getting started with GitHub Actions

With the repository structure in a good place, the next step is to bring automation into the workflow, and GitHub Actions makes that remarkably accessible. Since it is built right into GitHub, there is no need to set up additional servers or external tools. The automation lives alongside the code, watching for changes and reacting immediately whenever someone pushes updates or opens a pull request.

GitHub Actions uses simple configuration files to define what should happen automatically. These files sit inside a hidden folder in the repository and describe instructions such as installing dependencies, running tests, or deploying a new version of the model. The best part is that developers do not need to worry about managing machines. GitHub provides runners that execute all the steps from a clean environment each time, ensuring that the application does not depend on what happened during a previous run.

The idea may feel unfamiliar at first, but the essence is simple. A workflow is a set of tasks that run on events. If new code is pushed, a workflow might trigger to check that data preprocessing still works. If a pull request is opened, another workflow might train the model on a smaller dataset to make sure the accuracy has not dropped. In many ways, GitHub Actions acts like a smart assistant, constantly inspecting the project and helping the team maintain quality without extra effort.

One of the strengths of GitHub Actions is how flexible it can be. It supports complicated pipelines, scheduled model retraining, and deployment to different platforms without leaving the GitHub